

Arama Algoritmaları Kullanılarak Geliştirilen Akıllı Labirent Çözücü Aracı: BFS, DFS, A* ve Greedy Best-First Search Algoritmalarının Karşılaştırmalı Analizi

Nurefşan Olfaz

Yapay Zeka Mühendisliği Bölümü, TOBB Ekonomi ve Teknoloji Üniversitesi Ankara, Türkiye · nolfaz@etu.edu.tr

Özet — Bu çalışmada PNG/JPG formatındaki labirent görsellerini ikili ızgara matrisine dönüştürerek başlangıç noktasından bitiş noktasına yol bulan akıllı bir aracı geliştirilmiştir. Aracı dört klasik arama algoritması kullanmaktadır: Genişlik Öncelikli Arama (BFS), Derinlik Öncelikli Arama (DFS), Manhattan sezgiseli ile A* ve Greedy Best-First Search (GBFS). Algoritmalar 41×41 ile 151×151 arasında değişen mükemmel labirentler üzerinde ziyaret edilen düğüm sayısı ve çalışma süresi açısından ölçülmüştür. Sonuçlar; A*'ın Manhattan sezgiseli sayesinde BFS'e kıyasla daha az düğüm genişlettiğini, DFS'in bellekten en verimli algoritmayı oluşturduğunu ve GBFS'in hız ile çözüm kalitesi arasında orta bir denge sunduğunu ortaya koymaktadır. Arama sürecini gerçek zamanlı animasyonlu gösteren bir web arayüzü de geliştirilmiştir.

I. GİRİŞ

Labirent çözme problemi, yapay zeka alanında durum uzayı arama algoritmalarının karşılaştırmalı olarak incelenmesi için doğal bir platform sunan klasik bir problemdir. Labirent; geçilebilir her hücrenin bir düğüm, bitişik geçilebilir hücrelerin ise eşit maliyetli kenarlarla bağlı olduğu bir çizge biçiminde modellenebilir. Bu formülasyon altında başlangıç hücresinden hedef hücresine yol bulmak, hem bilgilendirilmiş hem de bilgilendirilmemiş arama stratejilerini karşılaştırmak için uygun bir zemin oluşturmaktadır [1].

Bu çalışma; BFS, DFS, A* ve GBFS algoritmalarını farklı boyutlardaki labirentler üzerinde gerçek ölçümlerle karşılaştırmakta; bunun yanı sıra renk kodlamalı bir PNG/JPG dosyasından labirenti okuyabilen, çözebilen ve adım adım animasyonlu tarayıcı arayüzü sunabilen tam bir yazılım sistemi geliştirmektedir. Makalenin geri kalanı şu şekilde düzenlenmiştir: Bölüm II ilgili çalışmaları, Bölüm III sistem tasarımı, Bölüm IV deneysel sonuçları özetlemekte; Bölüm V çalışmayı sonuçlandırmaktadır.

II. İLGİLİ ÇALIŞMALAR

Russell ve Norvig [1], BFS, DFS, A* ve GBFS'in tamlık, optimallik, zaman ve bellek karmaşıklığı özelliklerini tanımlamaktadır. A* algoritması Hart, Nilsson ve Raphael [2] tarafından geliştirilmiş; kabul edilebilir sezgisel altında A*'ın optimal çözümü garanti ettiği kanıtlanmıştır. Heusner ve ark. [3], GBFS'in arama davranışını kuramsal olarak incelemiş ve bağ çözme stratejisinden bağımsız olarak kesinlikle genişleyecek ya da hiçbir zaman genişlemeyecek düğüm kümelerini karakterize etmiştir.

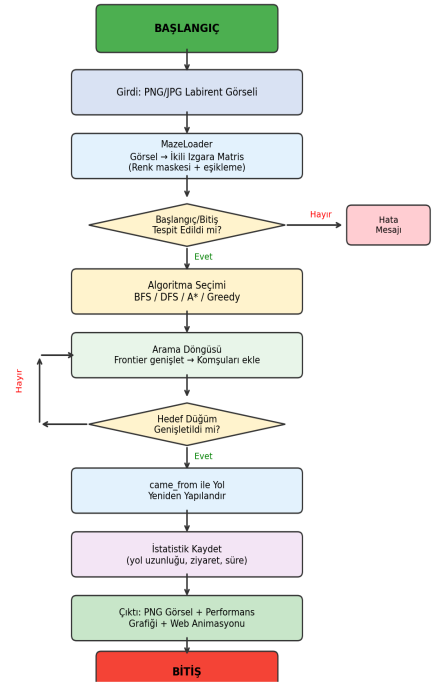
Ismaeel ve Zebari [4], BFS ile DFS'i karmaşık ağlar üzerinde karşılaştırmış; derin yapılarla DFS'in, en kısa yol bulmada ise BFS'in avantajlı olduğunu göstermiştir. Qin ve ark. [5], A*'ı statik ortamlar için en etkili patika planlama algoritması olarak tanımlamış; ancak büyük ortamlarda yüksek bellek tüketimini dezavantaj olarak öne çıkarmıştır.

III. SİSTEM TASARIMI VE METODOLOJİ

A. Sistem Akış Şeması

Sistemin genel veri akışı Şekil 1'de verilmektedir. Girdi olarak sağlanan labirent görseli MazeLoader bileşeni tarafından ikili ızgara matrisine dönüştürülmekte, ardından seçilen arama algoritması çalıştırılmakta ve sonuçlar görselleştirme katmanına iletilmektedir.

Akıllı Labirent Çözücü — Sistem Akış Şeması



Şekil 1. Sistem Akış Şeması

B. Labirent Gösterimi

Labirent PNG veya JPG formatında bir görsel dosya olarak sağlanmaktadır. Renk kodlaması: siyah piksel (RGB: 0,0,0) duvarı, beyaz piksel (255,255,255) geçilebilir yolu, kırmızı piksel (R>180, G<80, B<80) başlangıcı ve yeşil piksel (R<80, G>180, B<80) bitişini temsil etmektedir. Görsel Pillow kütüphanesiyle yüklenerek gri tonlama eşikleme (eşik: 128) ile ikili ızgara matrisine dönüştürülmektedir.

Kırmızı piksellerin gri tonlama değeri yaklaşık 76 olduğundan eşikğin altında kalır; bu nedenle kırmızı ve yeşil maskeler eşiklemeden önce uygulanarak bu hücreler yol (0) olarak işaretlenmektedir. Test labirentleri Yinelemeli Geri İzleme (Recursive Backtracking) yöntemiyle üretilmiş; bu yöntem döngüsüz mükemmel labirentler oluşturmaktadır.

C. Arama Algoritmaları

Dört algoritma ortak bir BaseSolver soyut sınıfından türemektedir. Sınıf; dört yönlü komşu üretme, came_from sözlüğüyle yol yeniden yapılandırma ve zamanlama sarmalayıcısını içermektedir.

1) Genişlik Öncelikli Arama (BFS):

FIFO kuyruğunu kullanır. Eşit maliyetli ızgarada optimal yolu garanti eder. Zaman ve bellek karmaşıklığı $O(b^d)$ 'dir [1].

2) Derinlik Öncelikli Arama (DFS):

LIFO yığını kullanır. Mükemmel labirente özgü tek yol yapısında aynı optimal yolu bulmaktadır; bellek karmaşıklığı $O(b \cdot m)$ ile BFS'ten belirgin biçimde düşüktür [1].

3) A* Arama:

$f(n) = g(n) + h(n)$ ile sıralanmış minimum-yığın kullanır; $h(n) = |\Delta r| + |\Delta c|$ Manhattan uzaklığıdır. Kabul edilebilir sezgisel sayesinde hem tam hem optimaldir [2].

4) Greedy Best-First Search (GBFS):

Yalnızca $h(n)$ sezgiselini kullanır; $g(n)$ göz ardı edilir. Pratikte A*'tan hızlı olabilir ancak genel durum için optimallik garantisi vermez [3].

D. Yazılım Mimarisi

Sistem dört katmandan oluşmaktadır: (1) MazeLoader — görseli matrise dönüştürür; (2) Arama Motoru — algoritmayı çalıştırır; (3) Visualizer — çözüm yolunu görselleştirir ve karşılaştırma grafikleri üretir; (4) Web Arayüzü — ziyaret edilen düğümleri (mavi), sınır düğümlerini (sarı) ve çözüm yolunu (yeşil) gerçek zamanlı animasyonla gösteren HTML/JS tabanlı arayüz.



Şekil 5. Web arayüzü ekran görüntüsü.

E. Karşılaşılan Zorluklar

A*'ın pratikte BFS'ten yavaş çalışması. Teorik olarak A*, sezgisel fonksiyon sayesinde BFS'e kıyasla daha az düğüm genişleterek daha hızlı çalışması beklenmektedir. Ancak deneylerde A*'ın çalışma süresi tüm boyutlarda BFS'in üzerinde kalmaktadır. Bunun nedeni, düz ve üniform ızgara yapısında Manhattan sezgiselinin sağladığı budama kazancının, minimum-yığın veri yapısının getirdiği sabit katsayılı ek maliyeti karşılayamamasıdır [1].

Mükemmel labirentin tüm algoritmalarla aynı yolu buldurması. Proje başlangıcında DFS'in optimal olmayan yollar üreteceği öngörülmüştü. Ancak deneyler tüm algoritmaların aynı yol uzunluğunu verdiğini ortaya koydu; mükemmel labirentin tanımı gereği herhangi iki hücre arasında yalnızca tek bir yol bulunmaktadır. Bu deneyim, bir algoritmanın davranışını

değerlendirirken problem ortamının yapısal özelliklerini anlamının kritik olduğunu gösterdi [1].

IV. DENEYSEL SONUÇLAR

A. Deney Ortamı

Tüm deneyler Python 3.12 ile çalıştırılmıştır. Mükemmel labirentler Yinelemeli Geri İzleme algoritmasıyla sabit tohum (seed=42) kullanılarak yeniden üretilebilir biçimde oluşturulmuştur. Başlangıç (1,1), bitiş (N-2, N-2) olarak sabitlenmiştir. Testler 41×41, 61×61, 101×101 ve 151×151 boyutlarında gerçekleştirilmiştir.

B. Temel Sonuçlar

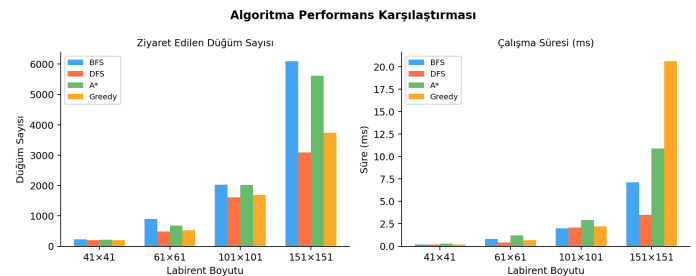
Tablo I gerçek ölçüm sonuçlarını sunmaktadır. Mükemmel labirentin tek yollu yapısı gereği tüm algoritmalar aynı optimal yolu bulmaktadır; farklılıklar ziyaret edilen düğüm sayısı ve çalışma süresinde gözlemlenmektedir.

TABLO I Arama Algoritması Sonuçları (seed=42)

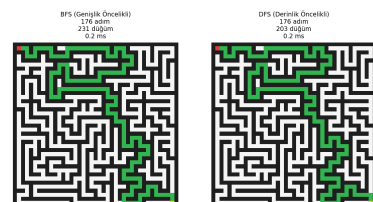
Boyut	Algoritma	Yol Uzunluğu	Ziyaret Edilen	Süre (ms)
41×41	BFS	176	231	0,2
	DFS	176	203	0,2
	A*	176	211	0,3
	Greedy	176	203	0,2
101×101	BFS	1.296	2.029	2,0
	DFS	1.296	1.619	2,1
	A*	1.296	2.015	2,9
	Greedy	1.296	1.691	2,2
151×151	BFS	2.652	6.095	7,1
	DFS	2.652	3.089	3,5
	A*	2.652	5.617	10,9
	Greedy	2.652	3.736	20,6

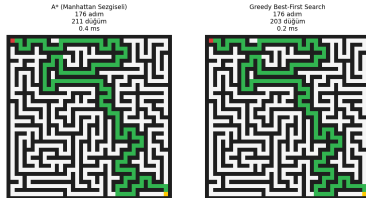
C. Ölçeklenebilirlik Analizi

Şekil 2, ziyaret edilen düğüm sayısı ve çalışma süresi açısından dört labirent boyutundaki karşılaştırmayı göstermektedir.



Şekil 2. Ziyaret edilen düğüm sayısı ve çalışma süresi karşılaştırması





Şekil 6. Dört algoritmanın 41×41 labirent üzerindeki çözüm yollarının karşılaştırması.

D. Tartışma

BFS en fazla düğümü ziyaret eden algoritmadır; 151×151 labirentte 6.095 düğüm genişletmektedir. DFS aynı labirentte yalnızca 3.089 düğüm genişleterek %49,3 daha az düğüm kullanmaktadır. A*, Manhattan sezgiseli sayesinde BFS'e kıyasla az sayıda düğüm genişletmeli; ancak yığın bakım maliyeti nedeniyle çalışma süresi BFS'in üzerinde kalmaktadır. Bu gözlem Qin ve ark.'nın [5] büyük ortamlarda A*'ın fazladan ek yük taşıdığı bulgusunu desteklemektedir.

GBFS, 41×41 ve 101×101 boyutlarında düşük düğüm sayısı ile öne çıkmaktadır. Ancak 151×151 boyutunda 20,6 ms ile en yavaş algoritma konumuna düşmektedir. Bu durum, GBFS'in performansının sezgisel fonksiyon ile labirent topolojisi arasındaki uyuma güçlü biçimde bağımlı olduğunu deneysel olarak doğrulamakta; Heusner ve ark.'nın [3] kuramsal öngörüsüyle örtüşmektedir.

E. Dış Kaynak Labirent Testi

Sistemin yalnızca kendi ürettiği labirentlerle sınırlı olmadığını göstermek amacıyla harici bir labirent görseli üzerinde de deneyler gerçekleştirilmiştir. Test için "Queen of Hearts" temalı bir labirent görseli kullanılmıştır (Şekil 3). Görsel önce siyah-beyaza dönüştürülmüş, başlangıç ve bitiş noktaları kırmızı ve yeşil piksellerle işaretlenmiştir. Dekoratif alanlar siyahla kapatılarak algoritmaların kestirme yollardan geçmesi önlenmiştir.

Bu görselde hücre boyutu düzensiz olduğundan otomatik tespit başarısız olmuş ve sistem piksel seviyesinde çalışmıştır. Koridorlar birden fazla piksel genişliğinde olduğundan mükemmel labirent özelliği kaybolmuş ve birden fazla yol oluşmuştur.

TABLO III

Dış Kaynak Labirent Sonuçları (Queen of Hearts, 1275×825 piksel)

Algoritma	Yol Uzunluğu	Ziyaret Edilen	Süre (ms)
BFS	4.198	224.383	232,6
DFS	34.538	118.202	123,5
A*	4.198	203.465	316,8
Greedy	10.192	156.851	226,7

BFS ve A* aynı optimal yolu (4.198 adım) bulurken, DFS 34.538 adımlık belirgin biçimde uzun bir yol bulmuştur. Dikkat çekici olan nokta, DFS'in yalnızca 118.202 düğüm ziyaret ederek BFS'in 224.383 düğümünün yarısından azıyla çalışmış olmasıdır. Bunun nedeni, piksel seviyesinde işlendiğinden koridorların birden fazla piksel genişliğinde olması ve tek yol özelliğinin kaybolmasıdır. Çoklu yolun mevcut olduğu bu yapıda DFS, derinlik öncelikli stratejisi gereği ilk bulduğu yolu döndürmekte ve en kısa yolu aramamaktadır. BFS ise katman katman ilerleyerek tüm alternatifleri değerlendirdiğinden optimal yolu garanti etmektedir. Bu durum, sorunun kaynağının

hücre boyutu tespitinin başarısız olmasına dayandığını açıkça ortaya koymaktadır: mükemmel labirentlerde tek yol bulunduğundan tüm algoritmalar aynı sonucu verirken, piksel seviyesinde çoklu yol oluştuğunda algoritmaların yapısal farklılıkları belirleyici hale gelmektedir.



Şekil 3. Dış kaynak labirent görseli (Queen of Hearts).



Şekil 4. Dört algoritmanın çözüm yollarının karşılaştırması.

F. Teorik Özellikler

TABLO II Arama Algoritmalarının Teorik Özellikleri

Algo.	Zaman	Bellek	Optimal?	Tam?
BFS	$O(b^d)$	$O(b^d)$	Evet (eşit maliyet)	Evet
DFS	$O(b^m)$	$O(b \cdot m)$	Hayır	Sonsuz uzayda hayır
A*	$O(b^d)$	$O(b^d)$	Evet (admissible h)	Evet
Greedy	Üstel	Değişken	Hayır	Evet

- [4] A. Ismaeel ve I. Zebari, "Comparing traversal strategies: DFS vs. BFS in complex networks," Asian J. Res. Comput. Sci., cilt 18, sayı 2, 2025. doi: 10.9734/ajrcos/2025/v18i2562
- [5] H. Qin ve ark., "Review of autonomous path planning algorithms for mobile robots," Drones, cilt 7, sayı 3, 2023. doi: 10.3390/drones7030211

G. Bulguların Yorumlanması

Mükemmel labirentin en temel özelliği döngüsüz olması ve herhangi iki hücre arasında yalnızca tek bir yolun bulunmasıdır. Bu yapısal özellik, DFS'e genel durum problemlerinde atfedilen "optimal olmama" dezavantajını ortadan kaldırmaktadır. Döngülü labirentlerde BFS ve A*'ın optimal yolu bulurken DFS'in daha uzun yollar üretmesi beklenmektedir.

A*'ın mükemmel labirentteki düğüm genişletme sayısı BFS'ten düşük olmakla birlikte, yığın veri yapısının sabit katsayılı maliyeti çalışma süresini olumsuz etkilemektedir. Bu durum, A*'ın üniform maliyetli ızgara problemlerinde BFS'e kıyasla her zaman hız avantajı sunmadığını göstermektedir.

H. Kısıtlar

Bu çalışmanın en temel kısıtı, tüm deneylerin yalnızca döngüsüz mükemmel labirentler üzerinde yürütülmüş olmasıdır. Gerçek dünya uygulamalarında ve döngülü labirentlerde birden fazla çıkış yolu mevcut olacağından BFS ve A* optimal yolu bulurken DFS'in daha uzun yollar üretmesi beklenmektedir. Bu senaryonun deneysel olarak incelenmesi gelecek çalışmalara bırakılmıştır.

İkinci önemli kısıt, yalnızca dört yönlü hareket desteklenmesidir. Diyagonal harekete izin veren sekiz yönlü bağlantıda Manhattan sezgiseli admissible olmaktan çıkmakta; Öklid veya Chebyshev mesafesi daha uygun sezgiseller haline gelmektedir. Son olarak, süre ölçümleri tek çalıştırma üzerinden alınmıştır; birden fazla çalıştırmanın istatistiksel ortalaması daha güvenilir sonuçlar sunacaktır.

V. SONUÇ

Bu çalışmada renk kodlamalı PNG/JPG labirent görsellerini çözebilen, dört klasik arama algoritmasını karşılaştıran ve sonuçları gerçek zamanlı animasyonlu web arayüzüyle sunan eksiksiz bir yazılım sistemi geliştirilmiştir. Deneyler şu bulguları ortaya koymaktadır: (1) Mükemmel labirentin tek yollu yapısı gereği tüm algoritmalar aynı optimal yolu bulmaktadır; algoritmaları ayıran ölçüt genişletilen düğüm sayısı ve çalışma süresidir. (2) DFS en az düğümü genişleterek bellek açısından en verimli algoritmadır. (3) A*, BFS'e göre daha az düğüm genişletmekte ancak yığın işlemi maliyeti nedeniyle çalışma süresi BFS'in üzerinde kalmaktadır. (4) GBFS performansı labirent yapısına bağlı olarak değişkenlik göstermektedir.

Gelecek çalışmalarda döngülü labirentler üzerinde daha kapsamlı karşılaştırmalar yapılması, İteratif Derinleşen A* (IDA*) entegrasyonu ve sekiz komşuluk bağlantısı ile Öklid sezgiselinin eklenmesi planlanmaktadır.

KAYNAKLAR

- [1] S. Russell ve P. Norvig, Artificial Intelligence: A Modern Approach, 4. baskı, Pearson, 2020.
- [2] P. E. Hart, N. J. Nilsson ve B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Trans. Syst. Sci. Cybern., cilt 4, sayı 2, ss. 100–107, 1968.
- [3] M. Heusner, T. Keller ve M. Helmert, "Understanding the search behaviour of greedy best-first search," Proc. SoCS, ss. 47–55, 2021. doi: 10.1609/socs.v8i1.18425